Application Development - Interacting with Data Objects - 1

Table of Contents

- Databases and their role in development
- Relational Databases and SQL
- SQL Primer - What you need to know
    - Database Connectors - SQLite, PGSQL, MS-SQL
- Getting Started with ADO.NET
    - Connecting to a database
    - Creating tables
    - Inserting Data
    - Retrieving Data

Databases and Development Role

Since the start of the semester until now, we have formed tables using arrays. We have built trees and graphs to model more complex connections and associations of data.

Databases within software development are an application of our data structures. They exist to suit particular workloads and enable persistence.

Databases

While commonly, databases are usually involved in web application development, they also exist in other roles where we may want to cache data, hold it between execution contexts and readily maintain data.

Being able to query and interact with a database is a crucial skill for retrieving data.

SQL Primer

For SQL, most of what you will be doing is going to be fairly standard between most SQL vendor options. However, we will utilise **PostgreSQL** as part of our training.

To sharpen your SQL skills, please use `https://tafe-tthom.github.io/sqlattack` application to work on some basics.

SQL Primer - Structure of SQL Queries

As a rough idea, we will prepare ourselves to understand SQL using the following format. It helps but we can see this model break when we create tables.

```
COMMAND_KEYWORD ([Identifiers/Expression])
([RELATED_KEYWORD (Identifiers/Expression)])
```

The above snippet probably doesn't mean much but it gives us a model of how sql statements are structured.

SQL Primer - Structure of SQL Queries

Let's take a look at a `SELECT` query.

```
SELECT firstname, lastname
FROM people
WHERE lastname = 'Smith'
```

We can observe that the `SELECT`, `FROM` and `WHERE` parts of the query are **keywords**.

The `firstname`, `lastname`, `people` components are identifiers.

The `lastname = 'Smith'` is an expression, in this case, narrowed down to be a predicate.

SQL Primer - Creating a Database

First action we will do is create a database on our newly created MariaDB server.

```
CREATE <RELATED_KEYWORD> <IDENTIFIER>
```

Let's create our library database.

```
CREATE DATABASE LibraryDB;
```

We can observe the structure of the statement and the creation of our database on our server.

SQL Primer - Creating and Using a Database

We now want to `use` our database.

```
use LibraryDB;
```

We specify this so, we know what database on the server we are using. Since a server can have multiple databases, we need to specify which one we want to use.

SQL Primer - Creating Tables

Now we can get on creating tables which is where our database comes alive.

```
CREATE TABLE Book(
    Id INT,
    ISBN CHAR(13) NOT NULL,
    Title VARCHAR(100) NOT NULL,
    Description VARCHAR(500) NULL,
    UnitPrice INT NOT NULL
)
```

The above sql query creates a table called `Book`, with five columns (Id, ISBN, Title, Description, UnitPrice).

SQL Primer - Columns

Tables have columns, each column will have an identifier and a datatype. Columns can be extended to do a lot more as we will see later.

Items inserted into a table are typically referred as **tuples** or **entries**.

SQL Primer - Column Data-Types

As what you noticed here is that each column has a description as part of its identifier.

```
    Id INT,
    ISBN CHAR(13) NOT NULL,
    Title VARCHAR(100) NOT NULL,
    Description VARCHAR(500) NULL,
    UnitPrice INT NOT NULL
```

So, let's break it down.

- Id, is an INT datatype, it means that accepts integer numbers (1, 2, 3... 100, -1, -2)
- ISBN is a CHAR and is also specified as NOT NULL, ISBN limit is that it will only accepted up to **13** characters. It also means it is exactly 13 characters. Any less and it pads it with '\0' characters.
- Title is a VARCHAR(100), it will accept up to 100 characters, but it will not pad.

SQL Primer - Inserting Records

One of the most important statements, we need to be able to insert data into our tables.

Assuming we were adding a new Product to a `Product` table

```
INSERT INTO Products (ProductName, SupplierID, CategoryId, Quantity, UnitPrice, UnitsInStock, UnitsOnOrder, ReorderLevel, discontinued)
VALUES('Super Soda', 30, 2, 5, 2, 10, 2, 5, FALSE);
```

SQL Primer - Inserting Records

We can insert multiple entries within the same statement as well.

```
INSERT INTO Products (ProductName, SupplierID, CategoryId, Quantity, UnitPrice,
UnitsInStock, UnitsOnOrder, ReorderLevel, discontinued)
VALUES
('Super Soda', 30, 2, 5, 2, 10, 2, 5, FALSE),
('Bad Cola', 30, 2, 7, 2, 10, 2, 5, TRUE),
('Old Bread', 30, 7, 7 2, 10, 2, 5, TRUE),
('Cake', 30, 3, 5, 2, 10, 2, 5, FALSE),
('Ice Cream', 30, 3, 5, 2, 10, 2, 5, FALSE),
```

SQL Primer - Selecting/Querying Data

You are able to retrieve elements from a table (and multiple tables) using the `SELECT` keyword.

Format:

```
SELECT [columns] FROM [tables]
```

There is also a wildcard symbol that allows the query to include all columns from a table (`*`).

SQL Primer - Selecting/Querying Data

However, lets start using some built-in functions within a query.

We will use the `CONCAT` function that allows us to merge two fields and represent it as a single column in the final results.

```
SELECT ID, Company AS [Company Name],
    CONCAT([LastName], ', ', [FirstName]) AS [Contact Person]
FROM Customers;
```

We can also specify the column name of these generated columns. The `AS` keyword is usable within this context.

SQL Primer - Selecting and Aliasing

The `AS` keyword is used as a way to give a column or result from an operation another name (alias). This is helpful if we want to have a shorthand for columns or tables.

```
SELECT C.ID, Company AS [Company Name] FROM Customers AS C;
```

In the above sample, we can observe how the keyword is applied here.

SQL Primer - Selecting and Where

You can constrain and search for a particular entries and leverage certain predicates for specific columns.

```
SELECT *
FROM Employees
WHERE ID >= 4 AND ID <= 9;
```

We are retrieving all employees that have an ID between 4 and 9.

SQL Primer - Selecting and Where

Negation of the field can be used to exclude entries from our final results.

```
SELECT *
FROM Customers
WHERE ID != 4;
```

SQL Primer - Selecting and Where

This can also take the form exclude a set or range of data.

```
SELECT *
FROM Employees
WHERE NOT (ID >= 4 AND ID <= 7);
```

This can be expanded to other data-types which will have their own semantics on operator usage. Please review the resource 8 at the end of this slide-deck.

ADO.NET - Data Objects Library

Usually when using ASP.NET we want to pair it with a database connect. This is usually in the form of using an ORM like **EntityFramework** but we will use the raw-connector api called **ADO.NET**.

ADO.NET - Using a Database

You can use the following sequelize module to interact with a SQL database. Without needing to know much about SQL, you can create a SQL database through C# by constructing the tables and entries through entity framework/ADO.NET. We will be using ADO.NET and a SQL controller to interact with a specific database.

It requires the following stages to be performed.

1. Initialise a connection to the database
2. Construct types with fields specified and the relevant types
3. Synchronise the model created with the database

ADO.NET - Database Connector - MySQL

Within this example we will use MySQL/MariaDB database as an example case. You can use MS-SQL, SQLite and PostgreSQL if you prefer but you will need to use a separate connector library.

With `dotnet`, you will need to add the package `MySqlConnector` using `dotnet add package MySqlConnect`. This library will then be added to your project that you then use within your ASP.NET project.

ADO.NET - Database Connection Demo

Inside your application you will maintain a way to connect and handle connections to a database. You will need to factor in appropriate credentials and roles for the application accessing this. However for this section, we will assume that we have already done this.

Let's look at the following snippet.

```
const string constring =
    "SERVER=localhost; DATABASE=Northwind;" +
    "UID=admin; PWD=somepassword";


var dbcon = new MySqlConnection(constring);
dbcon.Open();
string query = "SELECT * FROM Customers;";
var cmd = new MySqlCommand(query, dbcon);

var dataReader = cmd.ExecuteReader();

while (dataReader.Read())
{
    var idn = dataReader[0].ToString();
    var name = dataReader[1].ToString();

    Console.WriteLine($"{idn} - {name}");
}

dbcon.Close();
```

ADO.NET - Library Usage Breakdown

There is quite a bit to breakdown but we will start with the first two 5 lines of code. In the snippet below we have the connection string and the object that will form the database connection.

```
const string constring =
    "SERVER=localhost; DATABASE=Northwind;" +
    "UID=user; PWD=somepassword";


var dbcon = new MySqlConnection(constring);
dbcon.Open()
```

Once a connection is established, we can then manage it and terminate the connection using `.Close()`. Note that `constring` has a format where it is the address/hostname, database to be used, the username/id and the password to be used.

Once passed to the connection constructor, it will attempt to connect. Afterwards, if successful, we can then execute commands and receive results from this connection.

```
string query = "SELECT * FROM Customers;";
var cmd = new MySqlCommand(query, dbcon);

var dataReader = cmd.ExecuteReader();
```

The above snippet is the query we want to send to the server. We will keep it simple and send trivial select query to the server. The command must have the query string and the database connection. Afterwards, we are able to retrieve a `Reader` from the command, with the results.

```
while (dataReader.Read())
{
    var idn = dataReader[0].ToString();
    var name = dataReader[1].ToString();

    Console.WriteLine($"{idn} - {name}");
}
```

Once the reader has been retrieved, we can now process the results and assign it. In this example, we are just taking the first and second fields of the object we are reading and printing them out. However we are to construct objects or use the data however we like.

```
dbcon.Close()
```

Once the session has been finished, we should close the database connection. Use `.Close()` or more appropriately, use the connector within a `using` statement in which the resources will be released when needed.

ADO.NET - Writing Data

Taking the query snippet from above, we can insert data instead of reading. We want to ensure that do it in an appropriate way.

```
string query = "INSERT INTO Categories(CategoryID, CategoryName, Description) " +
  "VALUES(@cid, @cname, @cdesc);";

var cmd = new MySqlCommand(query, dbcon);
cmd.Parameters.AddWithValue("cid", "55");
cmd.Parameters.AddWithValue("cname", "Electronics");
cmd.Parameters.AddWithValue("cdesc", "All of the electronics!");

cmd.ExecuteNonQuery();
```

ADO.NET - Writing and Executing a Command

By accessing the `Parameters`, we are able to call `AddWithValue` which will associate the data matching the SQL format we have constructed. Each `@` component, will be its own variable that is then replaced with the value given.

ADO.NET - Database Connector - PostgreSQL

Within this example we will use MySQL/MariaDB database as an example case. You can use MS-SQL, SQLite and PostgreSQL if you prefer but you will need to use a separate connector library.

With `dotnet`, you will need to add the package `Npgsql` using `dotnet add package Npgsql`. This library will then be added to your project that you then use within your ASP.NET project.

ADO.NET - Database Connection Demo

Inside your application you will maintain a way to connect and handle connections to a database. You will need to factor in appropriate credentials and roles for the application accessing this. However for this section, we will assume that we have already done this.

Let's look at the following snippet.

```
const string constring =
    "Host=localhost;Database=Northwind;" +
    "Username=admin;Password=somepassword";


var dbcon = new NpgsqlConnection(constring);
dbcon.Open();



string query = "SELECT * FROM Customers;";
var cmd = new NpgsqlCommand(query, dbcon);

var dataReader = cmd.ExecuteReader();

while (dataReader.Read())
{
    var idn = dataReader[0].ToString();
    var name = dataReader[1].ToString();

    Console.WriteLine($"{idn} - {name}");
}

dbcon.Close();
```

ADO.NET - Library Usage Breakdown

There is quite a bit to breakdown but we will start with the first two 5 lines of code. In the snippet below we have the connection string and the object that will form the database connection.

```
const string constring =
    "SERVER=localhost; DATABASE=Northwind;" +
    "UID=user; PWD=somepassword";


var dbcon = new NpgsqlConnection(constring);
dbcon.Open()
```

Once a connection is established, we can then manage it and terminate the connection using `.Close()`. Note that `constring` has a format where it is the address/hostname, database to be used, the username/id and the password to be used.

Once passed to the connection constructor, it will attempt to connect. Afterwards, if successful, we can then execute commands and receive results from this connection.

```
string query = "SELECT * FROM Customers;";
var cmd = new NpgsqlCommand(query, dbcon);

var dataReader = cmd.ExecuteReader();
```

The above snippet is the query we want to send to the server. We will keep it simple and send trivial select query to the server. The command must have the query string and the database connection. Afterwards, we are able to retrieve a `Reader` from the command, with the results.

```
while (dataReader.Read())
{
    var idn = dataReader[0].ToString();
    var name = dataReader[1].ToString();

    Console.WriteLine($"{idn} - {name}");
}
```

Once the reader has been retrieved, we can now process the results and assign it. In this example, we are just taking the first and second fields of the object we are reading and printing them out. However we are to construct objects or use the data however we like.

```
dbcon.Close()
```

Once the session has been finished, we should close the database connection. Use `.Close()` or more appropriately, use the connector within a `using` statement in which the resources will be released when needed.

ADO.NET - Writing Data

Taking the query snippet from above, we can insert data instead of reading. We want to ensure that do it in an appropriate way.

```
string query = "INSERT INTO Categories(CategoryID, CategoryName, Description) " +
  "VALUES(@cid, @cname, @cdesc);";

var cmd = new MySqlCommand(query, dbcon);
cmd.Parameters.AddWithValue("cid", "55");
cmd.Parameters.AddWithValue("cname", "Electronics");
cmd.Parameters.AddWithValue("cdesc", "All of the electronics!");

cmd.ExecuteNonQuery();
```

ADO.NET - Writing and Executing a Command

By accessing the `Parameters`, we are able to call `AddWithValue` which will associate the data matching the SQL format we have constructed. Each `@` component, will be its own variable that is then replaced with the value given.